

Skriftlig prøve/dato: 27 maj 2020

Kursus navn: Maskinnær programmering

Kursus nummer: 02322

Hjælpemidler: Alle hjælpemidler

Varighed: 4 timer

Vægtning: Opgave 1–10%, Opgave 2–20%, Opgave 3–20%, Opgave 4–30%, Opgave 5–20%

Note: Hvis du mangler specifikationer, kan du definere dine egne og fortsætte opgaveløsningen. Ikke alle begreber/beskrivelser er oversat til dansk. Eksempelvis er ordene "run-time stack" og "pointer" ikke oversat. I enkelte tilfælde er ikke oversatte betegnelser indrammet i anførselstegn ("...").

Opgave 1 (10%)

Svar på følgende spørgsmål vedrørende talrepræsentation:

- a) Hvad er 2's komplement repræsentationen på 8 bit af den decimale tal 86?
- b) Hvad er 2's komplement repræsentationen på 8 bit af den decimale tal -86?
- c) Hvad er den hexadecimale repræsentation på 2 hexadecimale digit a den decimale tal 86?

Solution:

- a) 0101 0110
- b) 1010 1010
- c) x56

Opgave 2 (20%)

Givet nedenstående assemblerkode:

```
01          .ORIG x3000
02 func     ADD R6, R6, #-2
03          STR R7, R6, #0
04          ADD R6, R6, #-1
05          STR R5, R6, #0
06          ADD R5, R6, #-1
07          ADD R6, R6, #-1
08          LDR R0, R5, #4
09          LDR R2, R5, #5
10          LDR R1, R2, #0
11          NOT R1, R1
12          ADD R1, R1, #1
13          ADD R0, R0, R1
14          ADD R0, R0, #5
15          STR R0, R5, #0
16          BRnz label1
17          LDR R0, R5, #0
18          BRnzp label2
19 label1   AND R0, R0, #0
20 label2   STR R0, R5, #3
21          ADD R6, R5, #1
22          LDR R5, R6, #0
23          ADD R6, R6, #1
24          LDR R7, R6, #0
25          ADD R6, R6, #1
26          RET
```

Skriftlig prøve/dato: 27 maj 2020

Kursus navn: Maskinnær programmering

Kursus nummer: 02322

Hjælpemidler: Alle hjælpemidler

Varighed: 4 timer

Vægtning: Opgave 1–10%, Opgave 2–20%, Opgave 3–20%, Opgave 4–30%, Opgave 5–20%

Denne kode antages at være resultatet af en kompilering af en C funktion med navnet **func**.

Funktionen er kompileret med en LC3 kompiler som gør brug af en "runtime-stack" allokering.

- Angiv symboltabellen for denne kode.
- Oversæt assemblerkoden til C kode (d.v.s. skriv en C funktion **func** med same funktionalitet som assemblerkoden).

Solution:

a)

Symbol	Address
func	x3000
label1	x3011
label2	x3012

- First we should figure out what the assembly code does. We can do that by putting comments on the assembly code. This is a heavily commented version of the code:

```

.ORIG x3000
func ADD R6, R6, #-2 ;allocate space for return value and return address
STR R7, R6, #0 ;save return address on the stack
ADD R6, R6, #-1 ;push dynamic link (current frame pointer) on the stack
STR R5, R6, #0
ADD R5, R6, #-1 ;new frame pointer points at the start address for local
variables
ADD R6, R6, #-1 ;allocate space for 1 local variable (let's name it t)
LDR R0, R5, #4 ;R0 <- first argument of the function (let's name first
argument x)
LDR R2, R5, #5 ;R2 <- second argument of the function (let's name
second argument y)
LDR R1, R2, #0 ;R1=(*y) (R1 <- value store at the address pointed by y)
NOT R1, R1 ;R1 <- -R1 (R1 = -(y))
ADD R1, R1, #1
ADD R0, R0, R1 ;R0 <- R0 + R1 (R0 = x - (y))
ADD R0, R0, #5 ;R0 <- R0 + 5 (R0 = x - (y) + 5)
STR R0, R5, #0 ;set local variable (t = x - (y) + 5)
BRnz label1 ;if (t > 0)
LDR R0, R5, #0 ; then R0 <- t
BRnzp label2
label1 AND R0, R0, #0 ; else R0 <- 0
label2 STR R0, R5, #3 ;return R0
ADD R6, R5, #1 ;pop local variables from the stack
LDR R5, R6, #0 ;restore frame pointer and pop it from the stack
ADD R6, R6, #1
LDR R7, R6, #0 ;restore the return address and pop it from the stack
ADD R6, R6, #1
RET
.END

```

Skriftlig prøve/dato: 27 maj 2020

Kursus navn: Maskinnær programmering

Kursus nummer: 02322

Hjælpemidler: Alle hjælpemidler

Varighed: 4 timer

Vægtning: Opgave 1–10%, Opgave 2–20%, Opgave 3–20%, Opgave 4–30%, Opgave 5–20%

After we've understood the assembly code we can create the C function:

```
int func(int x, int* y)
{
    int t;
    t=x-(*y)+5;
    if (t>0)
        return t;
    else
        return 0;
}
```

Opgave 3 (20%)

Variablen z erklæres i nedenstående kode:

```
int main() {
    struct S1 {
        int a;
        char b;
        char* c;
    }* z;

    //Missing code 1

    //Missing code 2

    return 0;
}
```

- a) Hvad er størrelsen (i bytes) af variabelen z, givet at typen **int** er 4 byte, **char** er 1 byte og at adresser angives med 32 bit?

z is a pointer so it contains the address of a variable of the type Struct S1. Since addresses are represented on 32 bits then the size of x is 4 bytes.

- b) Udfyld funktionen **main**, lige efter kommentaren "Missing code 1" så z pege på en struct som du skal allokere på runtime-stacken hvor medlem variablerne har værdierne a=5, b='z', og hvor c pege på strengen "Pikaboo" som du skal søge for at allokere også på runtime-stacken.
- c) Udfyld funktionen **main**, lige efter kommentaren "Missing code 2" så vi bruger z til at ændre de a, b and c medlem variable af den struct z pege på, hvor:
- a skal få værdien 7
 - b skal få værdien 'b'
 - c skal pege på strengen "OK" som du skal søge for at allokere på heapen.

Skriftlig prøve/dato: 27 maj 2020

Kursus navn: Maskinnær programmering

Kursus nummer: 02322

Hjælpemidler: Alle hjælpemidler

Varighed: 4 timer

Vægtning: Opgave 1–10%, Opgave 2–20%, Opgave 3–20%, Opgave 4–30%, Opgave 5–20%

Solution for b & c:

```
int main() {
    struct S1 {
        int a;
        char b;
        char* c;
    }* z;

    //Missing code 1
    char s[]="Pikaboo";
    struct S1 aa = {5, 'z', s};
    z = &aa;
    //Missing code 2
    z->a = 7;
    z->b = 'b';
    z->c = (char*)malloc(10*sizeof(char));
    z->c[0]='O';
    z->c[1]='K';
    z->c[2]='\0';

    return 0;
}
```

Opgave 4 (30%)

Mikkel vil gerne vide, hvis han kaster en terning 10 gange, hvor mange gange kaster han 3 og hvor mange gange kaster han 4. En terning har værdier mellem 1 og 6. Hjælp Mikkel med at lave en LC-3 assembler program som løser problemet. Værdierne fra de 10 terningkast er allerede gemmet i hukommelsen fra adressen x4400 til adresse x4409 før programmet begynder at kører (du kan selv udfylde værdierne i LC3 simulator). Programmet skal tælle hvor mange gange der blev kastet 3 og hvor mange gange det blev kastet 4 og den vil vise resultanten som tekst på konsollen som i dette eksempel:

Hvis hukommelsen på adresserne x4400 til 4409 har værdierne:

3, 5, 2, 6, 4, 4, 2, 3, 1, 4

Beskeden vi får på skærmen er:

Terningen fik 2 gange værdien 3 og 3 gange værdien 4

Skriftlig prøve/dato: 27 maj 2020

Kursus navn: Maskinnær programmering

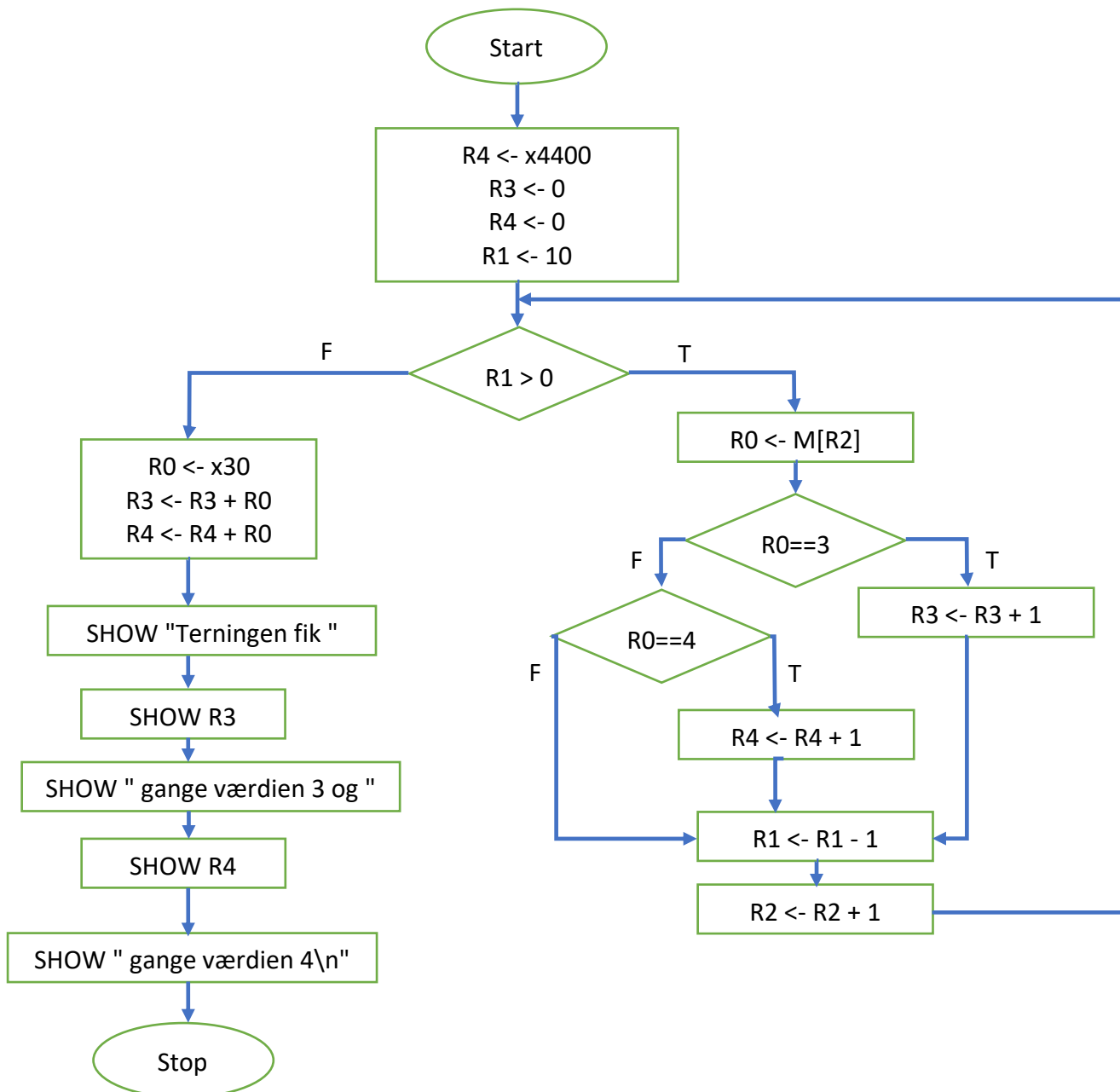
Kursus nummer: 02322

Hjælpemidler: Alle hjælpemidler

Varighed: 4 timer

Vægtning: Opgave 1–10%, Opgave 2–20%, Opgave 3–20%, Opgave 4–30%, Opgave 5–20%

a) Tegn et flowdiagram for programmet.



Skriftlig prøve/dato: 27 maj 2020

Kursus navn: Maskinnær programmering

Kursus nummer: 02322

Hjælpemidler: Alle hjælpemidler

Varighed: 4 timer

Vægtning: Opgave 1–10%, Opgave 2–20%, Opgave 3–20%, Opgave 4–30%, Opgave 5–20%

b) Skriv programmet i LC-3 assembler.

```

.ORIG x3000
LD R2, AR
AND R3, R3, #0
AND R4, R4, #0
ADD R1, R3, #10
SL BRnz MSG
LDR R0, R2, #0
ADD R5, R0, #-3
BRnp T3
ADD R3, R3, #1
BRnzp LB
T3 ADD R5, R0, #-4
BRnp LB
ADD R4, R4, #1
LB ADD R1, R1, #-1
ADD R2, R2, #1
BRnzp SL
MSG LD R0, AO
ADD R3, R3, R0
ADD R4, R4, R0
LEA R0, S1
PUTS
ADD R0, R3, #0
OUT
LEA R0, S2
PUTS
ADD R0, R4, #0
OUT
LEA R0, S3
PUTS
HALT

AR .FILL x4400
S1 .STRINGZ "Terningen fik "
S2 .STRINGZ " gange værdien 3 og "
S3 .STRINGZ " gange værdien 4\n"
AO .FILL x30
.END

```

Opgave 5 (20%)Følgende C program ønskes kørt på en **LC3 processor**:

```

#include <stdio.h>
int f(int *x){
    int a=5;
    struct {int a; int b;} s[3]={5,3},{1,6},{4, 0}};
    int* p;
    int b[3]={5,4,2};

```

Skriftlig prøve/dato: 27 maj 2020

Kursus navn: Maskinnær programmering

Kursus nummer: 02322

Hjælpemidler: Alle hjælpemidler

Varighed: 4 timer

Vægtning: Opgave 1–10%, Opgave 2–20%, Opgave 3–20%, Opgave 4–30%, Opgave 5–20%

```
p=&(s[1].b);
*(b+15) = 22;
s->b=21;
*(p-6) = 35;
b[2] = 7;
*(b+10) = 99;
(&s[0])->a = 0;
**(&p) = *x;
*(p+1) = 6;
a++;

return a;
}

int main() {
    int t=9;
    t=f(&t);
    return 0;
}
```

Hjælp til besvarelsen:

For en LC3 processor gælder:

- Integer og pointer variable er 16 bit (hvilket svarer til et ord i denne arkitektur)
- Hukommelsen er 16 bit (1 ord) adresserbar
- Lokale variable placeres på "runtime"-stakken i samme rækkefølge som de erklæres
- Rækkefølgen, hvormed argumenterne til et funktionskald placeres på stakken, går fra højre mod venstre

- a) Angiv indholdet af hele runtime-stakken gældende umiddelbart før udførelsen af "return a;" fra f funktionen. Det antages at stak-pointeren i R6 har værdien FFFF, når stakken er tom.

Skriftlig prøve/dato: 27 maj 2020

Kursus navn: Maskinnær programmering

Kursus nummer: 02322

Hjælpemidler: Alle hjælpemidler

Varighed: 4 timer

Vægtning: Opgave 1–10%, Opgave 2–20%, Opgave 3–20%, Opgave 4–30%, Opgave 5–20%

R6=>	FFEC	b[0]	5
	FFED	b[1]	4 -> 35
	FFEE	b[2]	2 -> 7
	FFEF	*p	FFF3 (&s[1].b)
	FFF0	s[0].a	5 -> 0
	FFF1	s[0].b	3 -> 21
	FFF2	s[1].a	1
	FFF3	s[1].b	6 -> 22 (*x)
	FFF4	s[2].a	4 -> 6
	FFF5	s[2].b	0
R5=>	FFF6	a	5 -> 100
	FFF7	Frame pointer	
	FFF8	Return address	
	FFF9	Return value	
	FFFA	*x	FFFB (&t)
	FFFB	t	
	FFFC	Frame pointer	
	FFFD	Return address	
	FFFE	Return value	
	FFFF		